CARAGA STATE UNIVERSITY
COLLEGE OF COMPUTING AND INFORMATION SCIENCES
CSU-Main Campus, Ampayon, Butuan City, Philippines
Competence Service Uprightness
YOUR BEST OPTION TO SUCCESS

ITE 18 – Application Development and Emerging Technologies

MIDTERM PROJECT DOCUMENTATION

FRONTEND TRACK

CAMPUSCART

A Campus Marketplace Frontend Prototype

Felix D. Garcia III

Mark Gabriel S. Pradel

Specialization: Frontend

ITE 18 – AQCK1

Ms. Glaicel A. Araneta

October 21, 2026

1st Semester, A.Y. 2026–2027

I. Project Overview

- **Project description**

  CampusCart is a frontend prototype of an online marketplace for a single university community. It lets students browse second hand or new items, such as textbooks, electronics, school supplies, fashion items and dorm essentials, offered by fellow students and approved campus vendors. Instead of shipping, buyers and sellers meet at public pickup points on campus.

  Problem addressed. Buying and selling on campus usually happens through scattered social media posts and group chats. Listings are hard to search, sellers are not reviewed, and there is no consistent way to report suspicious items or to arrange a safe meetup. CampusCart proposes one campus-only space with searchable listings, seller accounts that are reviewed by an administrator, a listing report feature and fixed campus pickup locations.

  Domain. E-commerce, specifically a campus community marketplace where students buy from and sell to each other.

- **Objectives**

  The frontend prototype aims to:

  1. Present a complete and consistent interface for three user groups: buyers (including guests), sellers and an administrator.
  2. Build the interface from reusable Vue 3 components written in TypeScript.
  3. Demonstrate the main marketplace workflows end to end in the browser: browsing, signing in, cart, checkout, seller application and administrator review.
  4. Keep shared application data in one central store that every screen reads from.
  5. Apply responsive layouts and basic accessibility practices to every implemented screen.
  6. Validate input on the main forms and show clear, specific error messages.
  7. Label every simulated action clearly so that viewers are never misled into thinking a real transaction took place.

- **Target users**

Table 1. *Target users and their needs*

User group	Who they are	What they need from the interface
Guest	Anyone who opens the site without signing in	Browse, search, filter and view product details without creating an account first.
Buyer	A registered member of the campus community (school or personal email)	Save items, use a cart, check out with campus pickup, track orders, read notifications and messages, edit a profile, and apply to sell.
Seller	A buyer whose seller application has been approved (student-owned store or campus vendor)	A dashboard, product listing and stock management, store order handling and store settings, while still being able to shop.
Administrator	Campus staff who moderate the marketplace	An overview of marketplace activity, seller application review, product report handling, and views of users, products and all orders.

- **Prototype scope**

  Table 2 separates what the interface implements from what is only simulated and what is outside the scope of this frontend prototype.

Table 2. *Implemented, simulated and not included features*

Area	Implemented in the interface	Simulated or mock	Not included (needs a backend)
Accounts and sign-in	Sign-in, account creation and sign-out forms with validation; guest access rules	Accounts are stored in the browser; mock passwords in plain text; no verification email	Real authentication service, password hashing, server sessions
Products	Browsing, search, category chips, filters, sorting, product details; seller add and edit	Eight seeded sample products; a sample image is used for new listings; selected photos are not uploaded	Product database and image storage
Cart and checkout	Cart with stock-limited quantities and totals; checkout form	"Place sample order" creates an order record in the browser; GCash is labelled simulated; no money is collected	Payment gateway
Orders	Status flow: Pending, Confirmed, Ready for Pickup, Completed or Cancelled	Status changes and notifications exist only in the browser	Email or push notifications
Seller application	Application form, status tracking, seller tools unlocked on approval	Approval is done by the demo administrator in the same browser	Document or permit verification

Area	Implemented in the interface	Simulated or mock	Not included (needs a backend)
Administrator	Dashboard, seller applications, product reports, users, products, all orders	Some dashboard totals are fixed mock numbers	Server-side role and access control
Messages	Chat screen with one sample conversation	Sent messages are added locally only	Real-time messaging
Campus sites	Campus picker in the header showing Main Campus	Other campuses are shown as "coming soon"	Campus site or location management

II. System Design & Development

> *Prototype status: CampusCart is an interactive user interface built for local preview and demonstration. It has no backend server, no production database, no real authentication service and no payment processing. Sign-in, checkout, seller approval and administrator access are simulated in the browser using sample (mock) data.*

A. System Architecture

- **Overview of frontend structure**

  CampusCart is a single-page application (SPA): the browser loads one HTML page once, and Vue then swaps the visible content as the user moves around, without loading new pages from a server. The application is organized in layers, shown in Figure 1:

  - **App shell** (`App.vue`) decides which view to display and whether the current user may see it.
  - **Layout components** (header, seller and admin sidebar, mobile menu and bottom navigation, footer) frame every page.
  - **Views** (`src/views/`) are one component per page, such as the marketplace, cart, checkout or seller dashboard.
  - **Reusable components and modals** (`src/components/`) are shared building blocks such as product cards, order cards, status badges and dialogs.
  - **Central store** (`useMarketplace.ts`) holds all shared data and actions. It loads its starting data from `demoData.ts` and saves changes to the browser through `persistence.ts`.

```
App.vue  (page switching + access rules)
├── AppHeader ── AppLogo, AccountActions ── ModeSwitch
├── <main>
│   ├── ProductDetailView                  (when a product is selected)
│   ├── Workspace layout (seller / admin)
│   │   ├── DashboardSidebar
│   │   └── DashboardView, ProductsView, OrdersView, PayoutsView,
│   │       ReviewsView, UsersView, VendorApprovalsView, ProductReportsView, ...
│   ├── Full-width pages: MarketplaceView, CartView, CheckoutView
│   └── Content pages:    ProfileView, SellView, MessagesView, NotificationsView
├── AppFooter, MobileNavigation (drawer + bottom bar)
└── Modals: AuthModals, ProductEditorModal, ReportModal, ReviewModal,
            SimpleModals (BaseModal), ToastNotification


Shared by every component:
  useMarketplace.ts (central store) ── demoData.ts (sample data)
                                    └─ persistence.ts (browser localStorage)
```

Figure 1. *Component hierarchy of the CampusCart prototype*

- **Framework used**

  The prototype uses **Vue 3** with the Composition API and `<script setup>` single-file components, written in **TypeScript** and bundled with **Vite**. Styling uses **Tailwind CSS v4** together with a custom stylesheet of design tokens. The versions declared in `package.json` are Vue ^3.5.13, TypeScript ~5.7.2, Vite ^6.3.5, Tailwind CSS ^4.1.0 and vue-tsc ^2.2.0 (for type checking); Node.js 18 or newer is required.

  Justification. Vue's single-file components keep each screen's template, logic and styles together, and its reactivity system makes shared state easy to follow. TypeScript catches mistakes in data shapes early, and Vite starts quickly and reloads changes instantly. The course outline prefers Nuxt. Plain Vue with Vite was chosen for the midterm because the prototype runs entirely in the browser and does not yet need Nuxt's server rendering or file-based routing. Because Nuxt is built on Vue 3, the existing components can be moved into a Nuxt project for the Final if the team chooses to.

B. UI/UX Design

- **Design principles applied**

  Color theory. The color palette is inspired by the CSU's official logo, which features green, yellow, and white. These colors were chosen to maintain consistency with the school's visual identity and create a familiar, cohesive look for CampusCart. Green represents growth, sustainability, and trust, aligning with the tagline "A little closer. A little greener." Yellow is used sparingly to highlight important elements and draw attention, while white and light sage backgrounds keep the interface clean, bright, and easy to navigate without distracting from product photos. Red is reserved for

error messages and destructive actions to ensure important warnings are clearly noticeable.

Table 3. *Color palette*

Color	Hex code	Where it is used
Primary green	#168447	Primary buttons, prices, active navigation, focus outline
Dark green	#105C32	Darker accents and hover states
Accent yellow	#F4C542	"Just for you" badge, pending counts, highlights
Ink	#202820	Main text and headings
Muted gray	#798079	Secondary text and descriptions
Sage backgrounds	#F0F6EB, #F4F8F1	Hero section and soft panels
Line	#E8ECE7	Borders and dividers
Red	#C4554D	Field errors, Cancel, Reject and Deactivate buttons

Typography. Headings use **Manrope** (weights 500 to 800) and body text uses **DM Sans** (weights 400 to 700), both from Google Fonts. The base size is 14 px with a line height of 1.5. Page titles are 36 px bold, section titles 25 px, and small uppercase labels (10 px, wide letter spacing) sit above headings to give context. Prices appear in extra-bold green Manrope so they stand out on every card.

Layout. Content sits in a centered container up to 1,280 px wide and uses CSS grid. Products appear in a four-column grid on desktop. Cart and checkout use two columns, with the items on the left and an order summary on the right. Seller and administrator pages share a workspace layout with a left sidebar. Every page starts with the same header pattern (label, title, short description). Cards share the same radius and shadow, empty lists show an icon, a message and a next action, and short toast messages confirm actions.

User-centered design. Guests can explore without creating an account and are asked to sign in only when they try a protected action, after which that action continues automatically. Every simulated step is labelled in the interface, for example "Place sample order", "GCash (simulated)" and "Demo only: no money is collected."

- **Wireframes, mockups, or prototypes**

  The interface was first designed as an interactive prototype in Figma Make ("CampusCart System Prototype") and then rebuilt as Vue components. The design covers the marketplace home, product details, sign-in and registration, cart, checkout, orders, profile, seller application, seller dashboard and tools, and the

administrator dashboard and review pages. The link to the design file is listed in the Appendix.

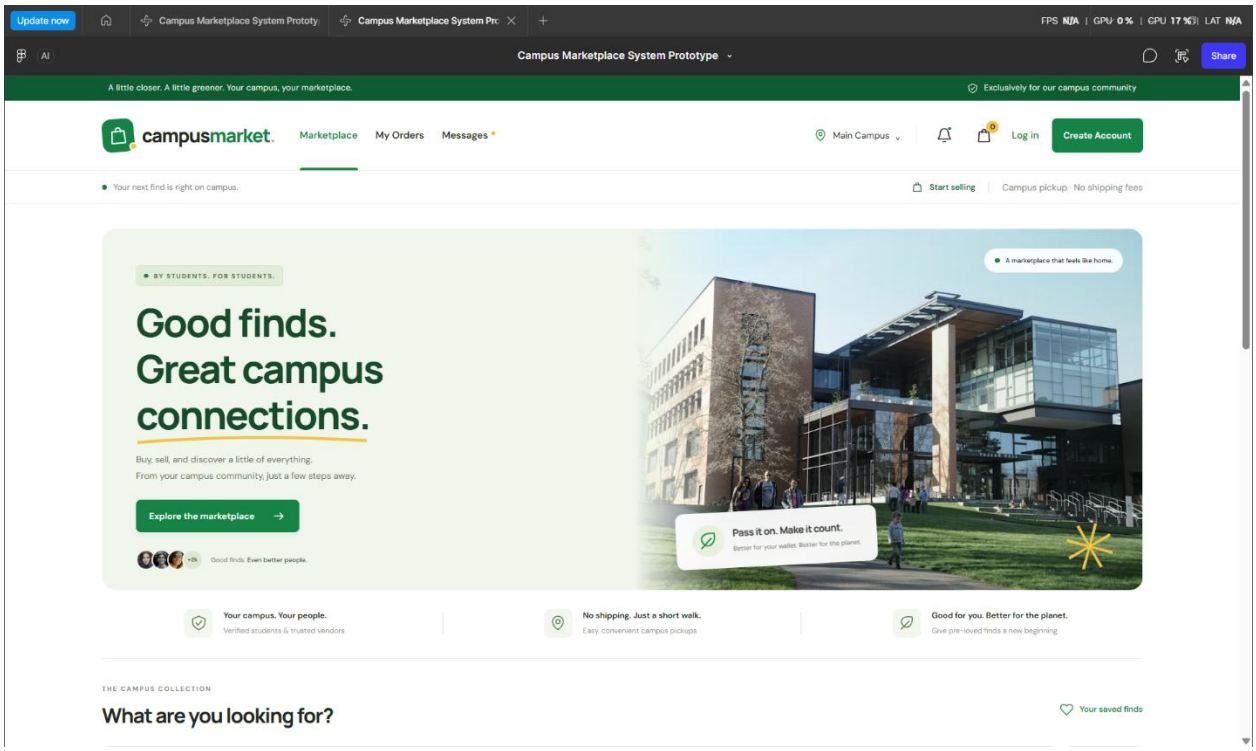


Figure 2. *CampusCart UI prototype in Figma Make*

- **Responsiveness strategy**

  The stylesheet uses width-based breakpoints. Layouts collapse from multi-column on desktop to a single column on phones, and navigation moves from the header to a slide-out menu and a bottom navigation bar.

Table 4. *Breakpoints and layout changes*

Screen width	Layout behavior
Above 1,150 px (desktop)	Four-column product grid, full header navigation, campus picker, sidebar workspace for sellers and the administrator
1,150 px and below (small laptop)	Campus picker hidden, narrower sidebar, statistic cards in two columns
900 px and below (tablet)	Three-column product grid; seller and admin sidebar replaced by the menu drawer; cart, checkout, dashboards and profile become single-column
640 px and below (mobile)	Two-column product grid; header links hidden; bottom navigation bar with five role-specific shortcuts
330 px and below	Single-column product grid

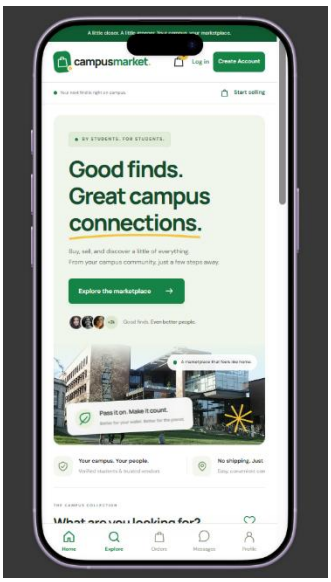


Figure 3. *Mobile layout of the marketplace*

- **User flows**

  The following workflows describe the main user journeys as they work in the prototype.

Workflow 1: Browsing as a guest

1. The guest opens the marketplace and sees the hero section, benefits and the product listing.
2. The guest types in the search box (matches product names and store names), picks a category chip, opens Filters to set a condition or maximum price, or changes the sort order.
3. The guest selects a product card to open its details page.

Workflow 2: Signing in before a protected action

1. A guest selects Add to cart, Buy now, the save (heart) button, Report this listing, or a page such as Orders, Messages or Profile.
2. The sign-in dialog opens and explains what will happen next, for example "Sign in to add this find to your cart. We'll add it right after."
3. The guest signs in with a demo account or creates a new account.
4. The prototype completes the original action once: the item is added to the cart, checkout opens, the item is saved, or the requested page opens.

Workflow 3: Buying an item (simulated)

1. The buyer adds items to the cart and adjusts quantities; quantities cannot exceed the seller's stock.
2. At checkout the buyer chooses Cash on pickup or GCash (simulated), a pickup location and a preferred pickup time.
3. Selecting Place sample order creates one order per store with status Pending and shows a confirmation that no payment was processed.
4. The seller confirms the order and marks it ready for pickup; the buyer then confirms pickup and may write a review.

Workflow 4: Applying to become a seller

1. A signed-in buyer selects Become a seller on the Profile page or in the bar below the header.
2. The buyer fills in the store application (store name, store type, main category, pickup spot, permit, description) and agrees to the selling guidelines.
3. The application status changes to Pending review, and a three-step tracker is shown.
4. After the demo administrator approves it, a Shopping / Selling switch appears and the seller dashboard becomes available. A rejected applicant can apply again.

Workflow 5: Reviewing a seller application (administrator)

1. The reviewer selects Demo admin below the header; the demo credentials are prefilled.
2. The admin dashboard shows the number of pending applications. The reviewer opens Seller Applications.
3. For a pending application, the reviewer selects Approve vendor or Reject application and confirms in a dialog.
4. The applicant's demo account is updated immediately and receives a notification.

C. Implementation Details (Progress to Date)

- **State management**

  The prototype does not use Vuex or Pinia yet. Instead, it has one central store, `useMarketplace.ts`, written with Vue's Composition API (`reactive`, `ref`, `computed` and `watch`). The store is created once at module level, so every component that calls `useMarketplace()` receives the same instance. In practice it plays the same role as a Pinia store, and moving it to Pinia is planned for the Final.

Table 5. *Contents of the central store*

Kind of state	Examples	Notes
Saved demo data	products, orders, reports, seller applications, accounts, per-account cart, saved items, messages and notifications, session, seller mode	Saved to localStorage under `campus-market-demo` about 120 ms after each change, and immediately when the tab is closed
Interface state	current page, selected product, search and filters, open dialog, toast message, mobile menu, pending sign-in intent	Not saved; reset on reload
Derived (computed) values	role (Buyer, Seller or Admin), filtered products, cart count and subtotal, unread notifications, visible orders, pending applications, open reports, low-stock products	Recalculated automatically when the data they depend on changes
Actions	login, register, adminLogin, logout, addToCart, buyNow, placeOrder, submitApplication, decideApplication, saveProduct, setOrderStatus, addReport	The only place where shared data is changed

Example feature: the shopping cart. The product details page calls `addToCart`, the cart badge in the header reads `cartCount`, and the cart page reads `cart` and `cartSubtotal`. Because all three read the same store, they update together. Saved data is validated field by field when it is loaded, so unreadable or outdated data falls back to the sample data instead of breaking the app. A "Reset demo data" button in the footer restores the starting state.

- **Component structure and routing**

Table 6. *Main reusable components*

Component	Purpose
AppHeader, AppFooter, AppLogo	Site header with role-based navigation, cart and notification icons; footer with reset button
AccountActions, ModeSwitch	Become a seller link, Shopping / Selling switch, Demo admin entry and Exit admin
DashboardSidebar, MobileNavigation	Seller and admin sidebar; mobile slide-out menu and bottom navigation bar
ProductCard, ProductGrid	Product tile (image, name, price, store, save button) and the grid that lists them
OrderCard, StatusBadge	Order summary with role-specific actions; colored status labels
StatCard, QuantityStepper	Dashboard statistic tile; plus and minus quantity control limited by stock

Component	Purpose
BaseModal, ToastNotification	Accessible dialog base used by all modals; short confirmation messages
AuthModals	Sign in, Create account and Demo admin access dialogs
ProductEditorModal, ReportModal, ReviewModal, SimpleModals	Add or edit product, report a listing, write a review, campus picker, approve or reject confirmation, order placed confirmation
AppIcon	Inline SVG line icons used throughout the interface

Routing. Vue Router is not used in the current version. Navigation is state-driven: the store keeps a `currentPage` value, and `App.vue` maps each page name to a view component. The store's `navigate` function applies the access rules: guests are asked to sign in, administrator pages open only through the demo admin entry, and seller pages require an approved application. `App.vue` also falls back to the marketplace if the current user may not see the current page, for example after signing out. As a result, the browser address does not change between pages, so individual pages cannot be bookmarked and the Back button does not move between them. Adding Vue Router is planned for the Final.

Table 7. *Pages implemented*

Page	View component	Who can open it
Marketplace (home)	MarketplaceView	Everyone
Product details	ProductDetailView	Everyone
Cart, Checkout	CartView, CheckoutView	Signed-in buyers
Orders	OrdersView	Buyers (own orders), sellers (store orders), administrator (all orders, read-only)
Messages, Notifications	MessagesView, NotificationsView	Signed-in buyers and sellers
Profile and settings	ProfileView (profile mode)	Signed-in users; shown as Settings for sellers
Become a seller	SellView	Everyone (guests see a sign-in prompt)
My Store, Payouts, Reviews	ProfileView (store mode), PayoutsView, ReviewsView	Approved sellers
Dashboard	DashboardView	Approved sellers and the administrator (different content)
Products	ProductsView	Sellers (own listings) and the administrator (all listings)

Page	View component	Who can open it
Users, Seller Applications, Product Reports	UsersView, VendorApprovalsView, ProductReportsView	Administrator only

- **Integration with backend or APIs**

  The prototype is not connected to a backend endpoint or to a mock API server such as JSON Server yet. All data comes from the sample data module `src/data/demoData.ts`, which defines 8 products, 4 demo accounts, 1 completed order, 1 product report and 2 seller applications. The store loads this data on first visit and afterwards reads and writes the saved copy in the browser's localStorage. Every list in the interface (products, orders, applications, users, reports) is read from this store.

  Product and hero photos are loaded from Unsplash and the hero avatars from pravatar.cc, so an internet connection is needed for images to appear.

- **Data validation and error handling**

  Forms use the `novalidate` attribute so the prototype can show its own messages. Each invalid field is marked with `aria-invalid`, and a red message appears directly below it. Account-level errors, such as an unknown email and password combination, appear at the top of the form in an alert region that screen readers announce.

Table 8. *Validation rules by form*

Form	What is validated
Sign in	Valid email format; password of at least 6 characters; the email and password must match a demo account; the admin email is refused in public sign-in
Create account	Full name and student number required; valid email; password of at least 6 characters; email not already registered; @campusmarket.demo addresses reserved
Checkout	Preferred pickup time required
Seller application	Store name of at least 3 characters; description of at least 10 characters; business permit required for campus vendors; agreement to the selling guidelines
Add or edit product	Name required; price of at least ₱1; stock must be a whole number of 0 or more
Profile and store settings	Name required; valid and unused email; at least one payment method for a store
Report listing, Write review	Details or review text required

Other error handling includes quantity limits based on stock (with a message when fewer items are available), empty-state messages for empty lists, a footer notice when the browser cannot save data, and a toast notice when saved data had to be reset.

D. Preliminary Testing and Debugging

- **Testing done so far**

  No automated unit or UI tests have been written yet. While this document was prepared (October 21, 2026), `npm run build` was run on the current source. It completed without errors: `vue-tsc` type-checked the project and Vite produced a production build of 89 modules.

  Table 9 lists manual test scenarios for the demo. Record the result of each scenario after running it.

Table 9. *Manual test scenarios*

#	Scenario	Expected result	Result
1	Search "calculus" and select the Books & Notes category as a guest	Only matching products are listed	Pass
2	Select Add to cart as a guest, then sign in with the demo buyer	Sign-in dialog opens; item is in the cart after sign-in	Pass
3	Clear the pickup time at checkout and submit	"Please choose a preferred pickup time." is shown	Pass
4	Place a sample order	Confirmation dialog appears; order shows as Pending in My Orders	Pass
5	Submit the seller application with an empty store name	Field error is shown and nothing is submitted	Pass
6	Approve a pending application in the demo admin, then sign back in as the applicant	Shopping / Selling switch and seller dashboard are available	Pass

- **Tools used**

  vue-tsc (TypeScript type checking), the Vite development server with hot reload, and the browser developer tools. The device toolbar in DevTools is useful for checking breakpoints, and the Application tab shows the saved `campus-market-demo` entry in localStorage.

- **Accessibility and responsiveness checks**

  The following accessibility features are present in the source code:

- A "Skip to content" link, and semantic header, nav, main and footer landmarks.
- Labels for all inputs (visually hidden where the design shows only an icon), and `aria-label` on icon-only buttons such as the cart, notifications and remove buttons.
- `aria-pressed`, `aria-expanded` and `aria-current` on toggles, the filter button and active navigation items.
- Dialogs marked `role="dialog"` with `aria-modal`, focus moved into the dialog, Tab kept inside it, Escape to close, and focus returned afterwards.
- A visible green focus outline, alt text on product images, live regions for chat and application status, and reduced motion when the user's system asks for it.

Contrast (calculated from the color tokens). Body text (#202820 on white) is 15.2:1. White text on the primary green is 4.75:1, which passes WCAG AA for normal text. Muted gray text (#798079 on white) is 4.06:1, which is below the 4.5:1 AA level for small text. Darkening this gray slightly is listed as a fix for the Final.

Devices and browsers tested: Laptop's browser and also mobile device view by inspecting the browser then setting it to mobile view.

III. Ethical and Professional Considerations

- **Ethical coding and data protection**

  No data leaves the user's browser. Accounts, carts and orders are stored only in that browser's localStorage. Because demo passwords are stored there in plain text, the sign-in and registration dialogs tell users to enter mock details only, and the README warns that the passwords must never be real. A production version would need server-side authentication and hashed passwords. Public registration always creates a buyer account and cannot create an administrator, and the administrator entry is kept separate from public sign-in. Every simulated action is labelled honestly ("Place sample order", "GCash (simulated)", "Paid (demo)", "This is a prototype. No payment was processed.").

- **Accessibility and inclusivity**

  Beyond the features listed in Section II.D, the prototype accepts both school and personal email addresses, so students without a school email can still join. Guests can browse without an account. The mobile layout and bottom navigation support students who mainly use phones, prices are shown in Philippine pesos, and the interface uses short, plain-language messages.

- **Copyright and attribution**

Table 10. *Sources and licenses of assets and libraries*

Asset or library	Source	License or terms
Vue, Vite, Tailwind CSS	npm packages	MIT License
TypeScript	npm package (Microsoft)	Apache License 2.0
DM Sans and Manrope fonts	Google Fonts	SIL Open Font License 1.1
Product and hero photographs	Unsplash (loaded by URL)	Unsplash License (free to use; attribution appreciated)
Hero avatar images	pravatar.cc placeholder service	Placeholder images for prototype display only
Icons	Inline SVG line icons defined in AppIcon.vue	Included in the project source
UI design	Team's Figma Make prototype	Original work of the team
Names, stores, reviews and orders	Sample data in demoData.ts	Fictional demo content

IV. Midterm Progress Report

This section presents each implemented screen with a short description. Screenshots are to be taken from the running prototype (`npm run dev`) using the demo accounts listed in the Appendix.

- **Implemented features and screenshots**

A. Public Marketplace

Homepage. The marketplace opens with an announcement bar, the header (logo, Marketplace / My Orders / Messages links, campus picker, cart and Sign in button) and a hero section with the message "Good finds. Great campus connections." and an "Explore the marketplace" button. Three benefit cards follow, then a "Start selling" banner at the bottom of the page.

Product listings, categories and search. The "What are you looking for?" section has a search box, a Filters panel (condition and maximum price), category chips (All items, Books & Notes, Electronics, Fashion & Accessories, School Supplies, Dorm Essentials, Others), a sort menu and the product grid. Each card shows the photo, name, price, store and a save button. If nothing matches, an empty state offers "Show all products".

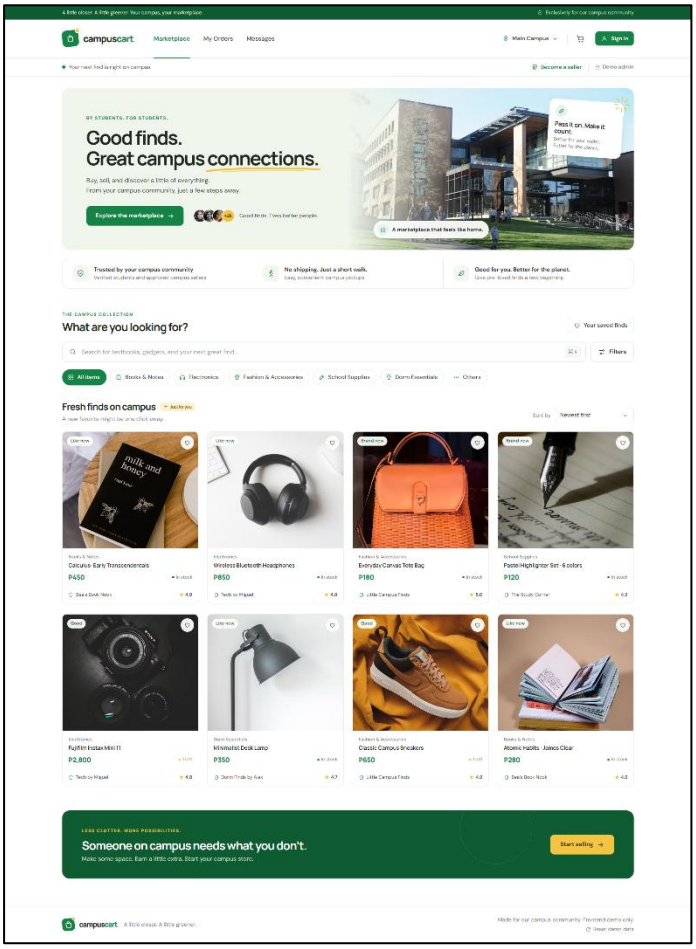


Figure 4. *CampusCart homepage with search, categories and product listings*

Product details. Selecting a card opens the details page: photo with save button, category, name, condition, price, seller panel with rating, quantity stepper with available stock, Add to cart and Buy now buttons, Message seller and Report this listing links, and a sample student comment. The description and comment are the same placeholder text for every product.

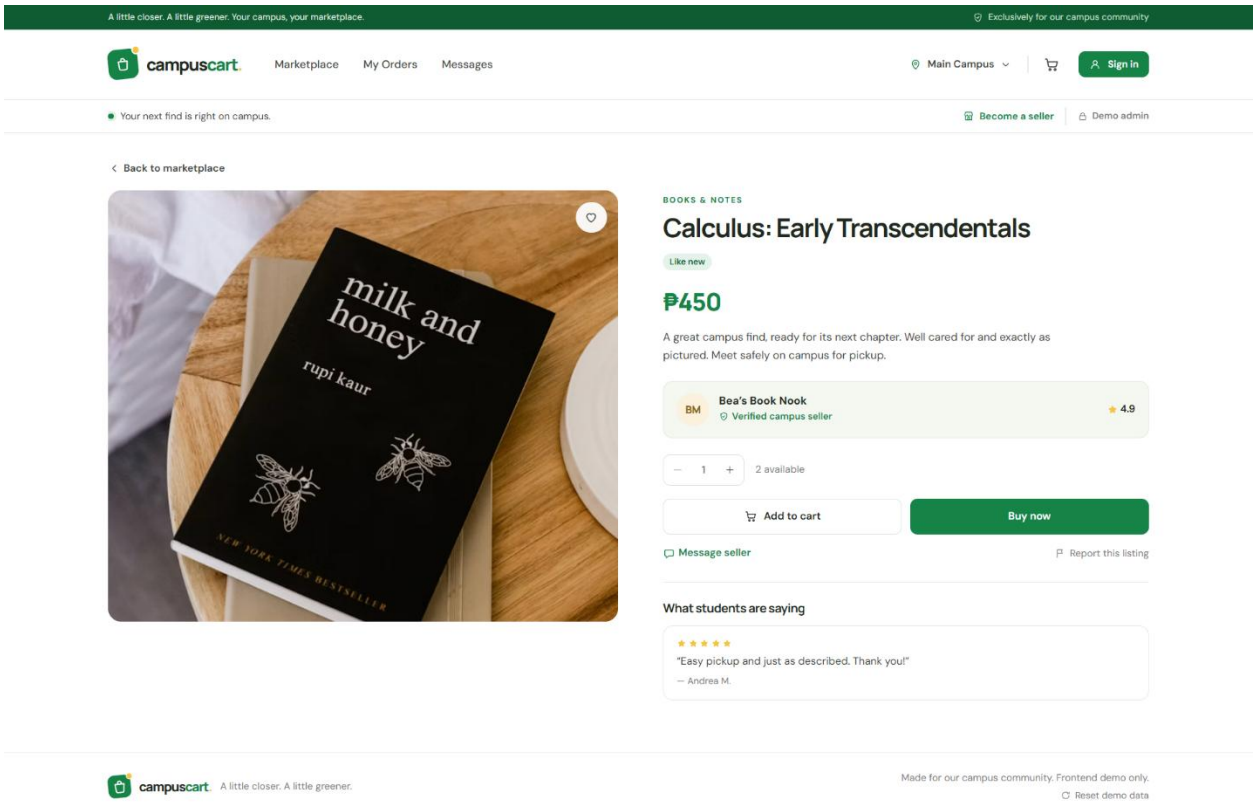


Figure 5. *Product details page*

B. Buyer Interface

Sign in and registration. The Sign in dialog accepts a school or personal email and a password, offers a "Use it" shortcut for the demo account, and states that no real authentication takes place. The Create account dialog collects full name, student number, department, campus, email and password. New accounts are always buyers.

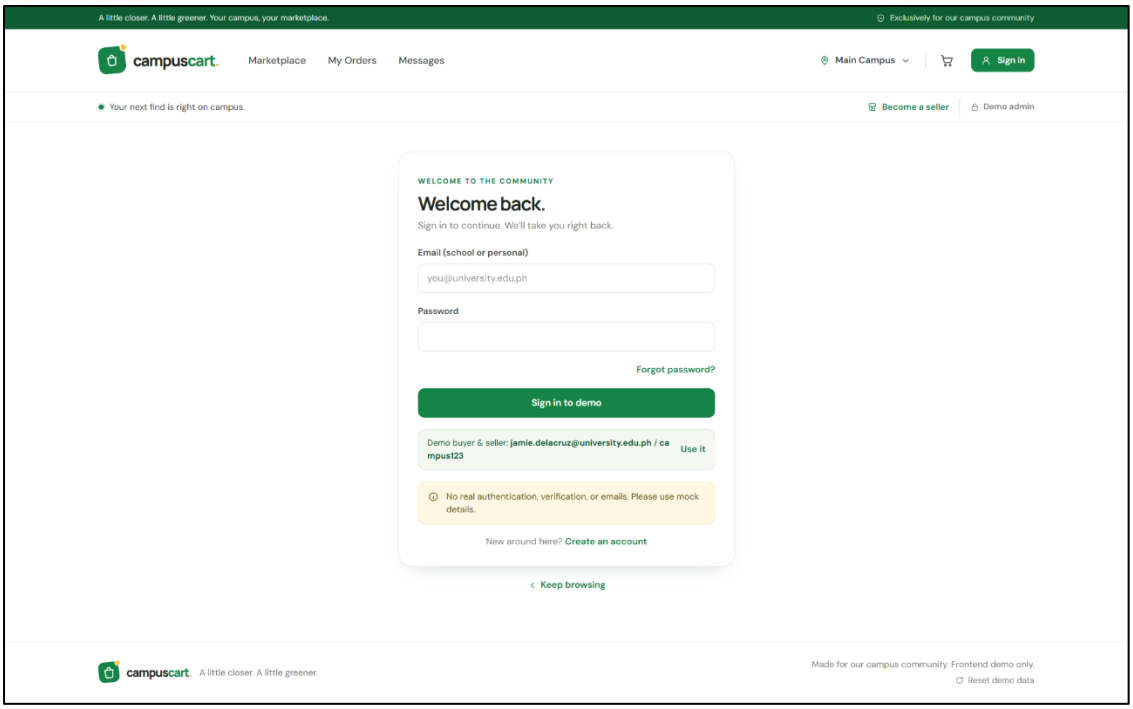


Figure 6. *Simulated sign-in and registration interface*

Shopping cart. The cart lists each item with its image, store, unit price, quantity stepper and remove button. The order summary shows the subtotal, a ₱0 platform fee, free campus pickup and the total, with Proceed to checkout and Continue shopping buttons.

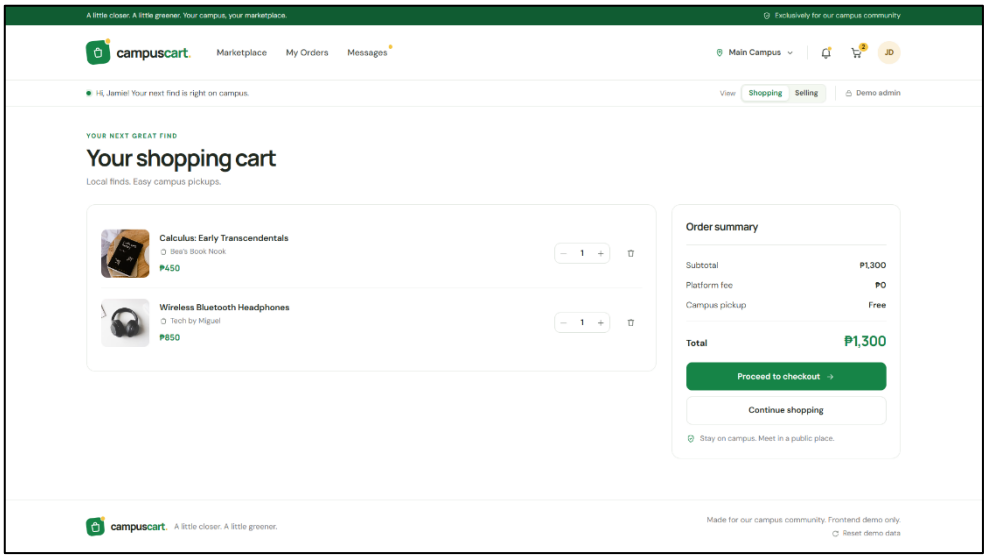


Figure 7. *Shopping cart with calculated totals*

Simulated checkout. Checkout shows the items under review, a payment choice (Cash on pickup or GCash (simulated)), the pickup location, the preferred pickup time and the order

summary. Place sample order creates one order per store and opens a "You're all set!" dialog stating that no payment was processed.

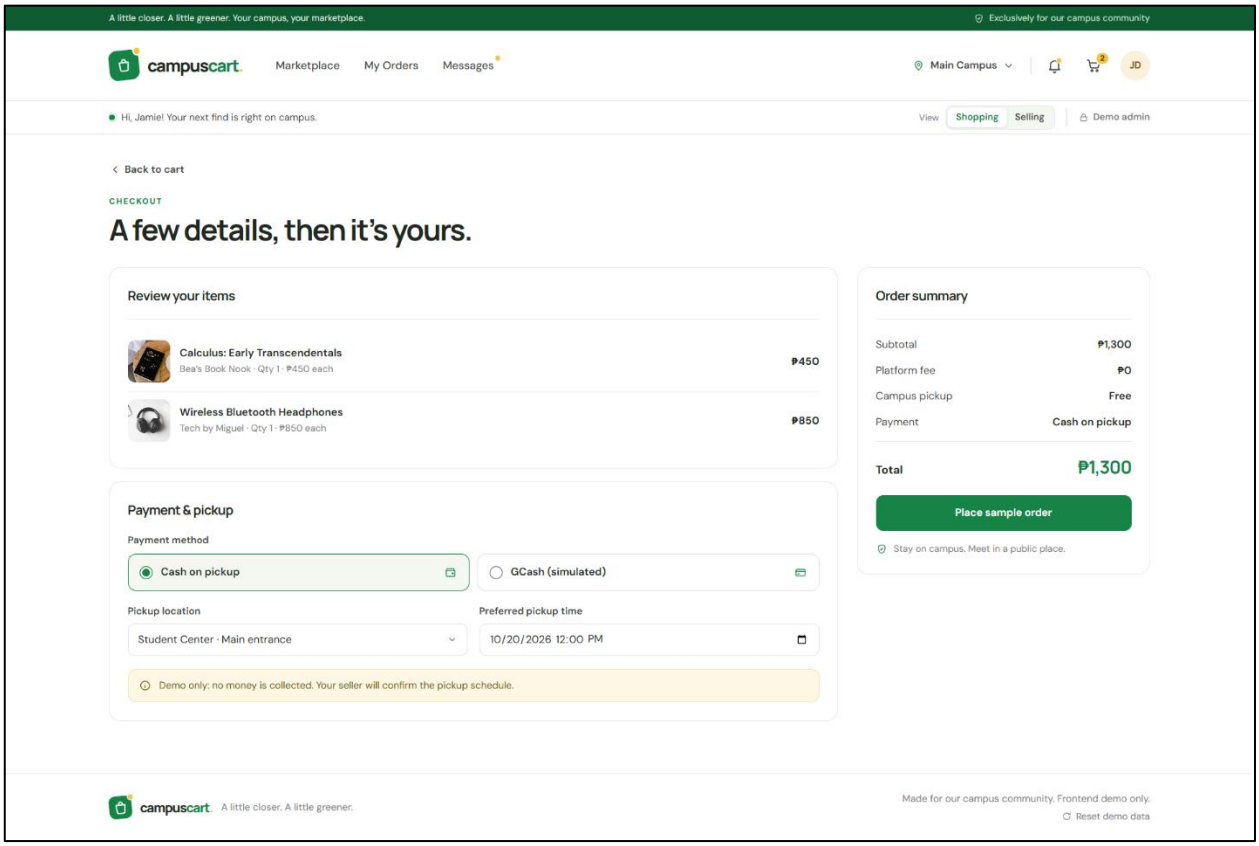


Figure 8. *Simulated checkout*

Order history. My Orders lists the buyer's orders with status tabs (All, Pending, Confirmed, Ready for Pickup, Completed, Cancelled). Each order card shows the items, pickup location and time, payment method and status, and the buyer can cancel a pending order, confirm pickup when it is ready, or write a review when it is completed.

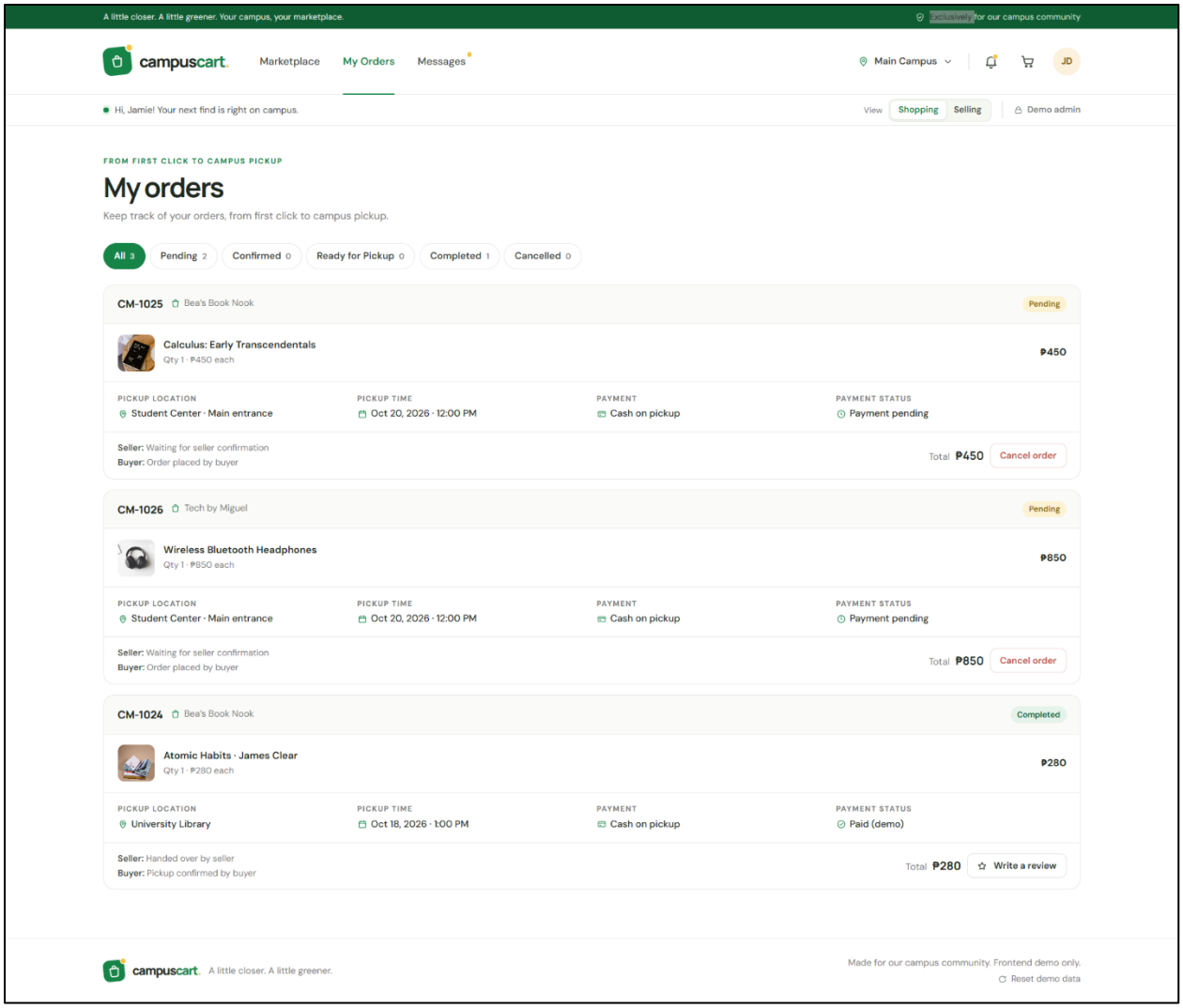


Figure 9. *Buyer order history*

Profile management. The Profile & settings page shows an account card (name, email, student number, department, campus and seller status) and an editable form for name, email, campus, department and notification preference. It also has a Sign out button. Buyers who are not yet sellers see a **Become a seller** button on the account card. Messages (one sample conversation) and Notifications (with Mark all as read) are also available to signed-in buyers.

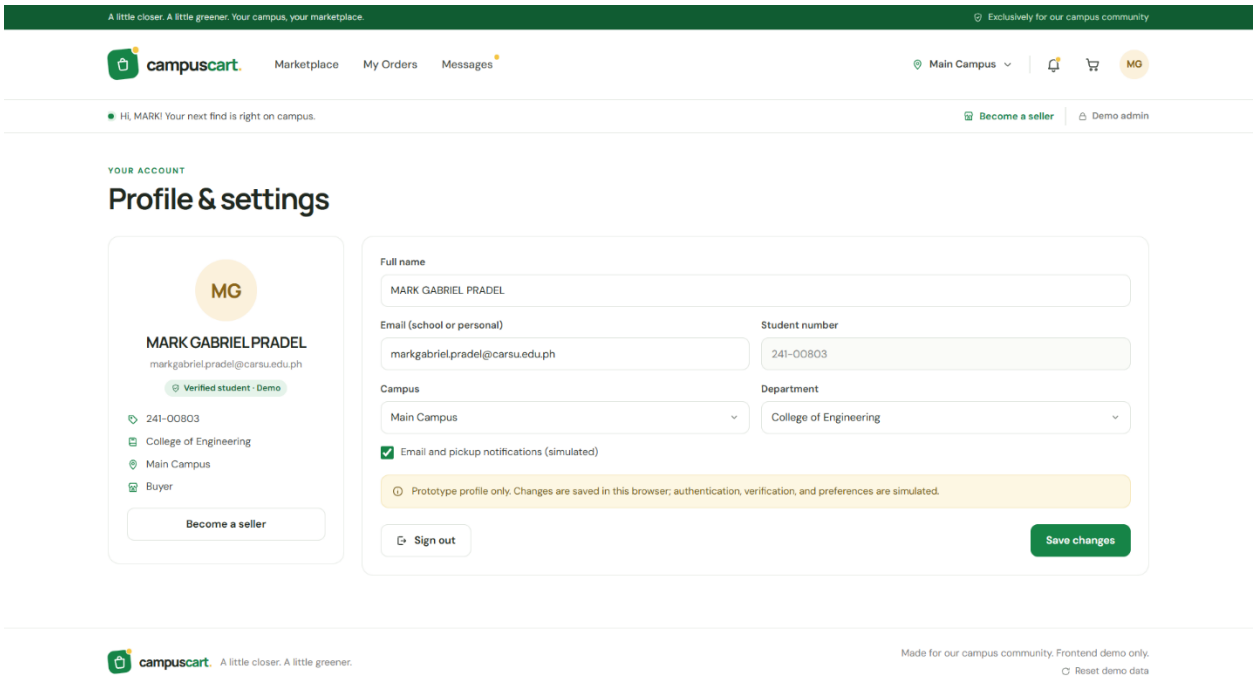


Figure 10. *Buyer profile with the Become a seller option*

C. Seller Application and Seller Dashboard

Seller application. The Become a campus seller page shows the application form (store name, store type, main category, usual pickup spot, business permit, description and guideline agreement) beside a "Why sell on campus?" panel. After submission, the page shows the status (Pending review, Approved or Not approved) with a three-step tracker.

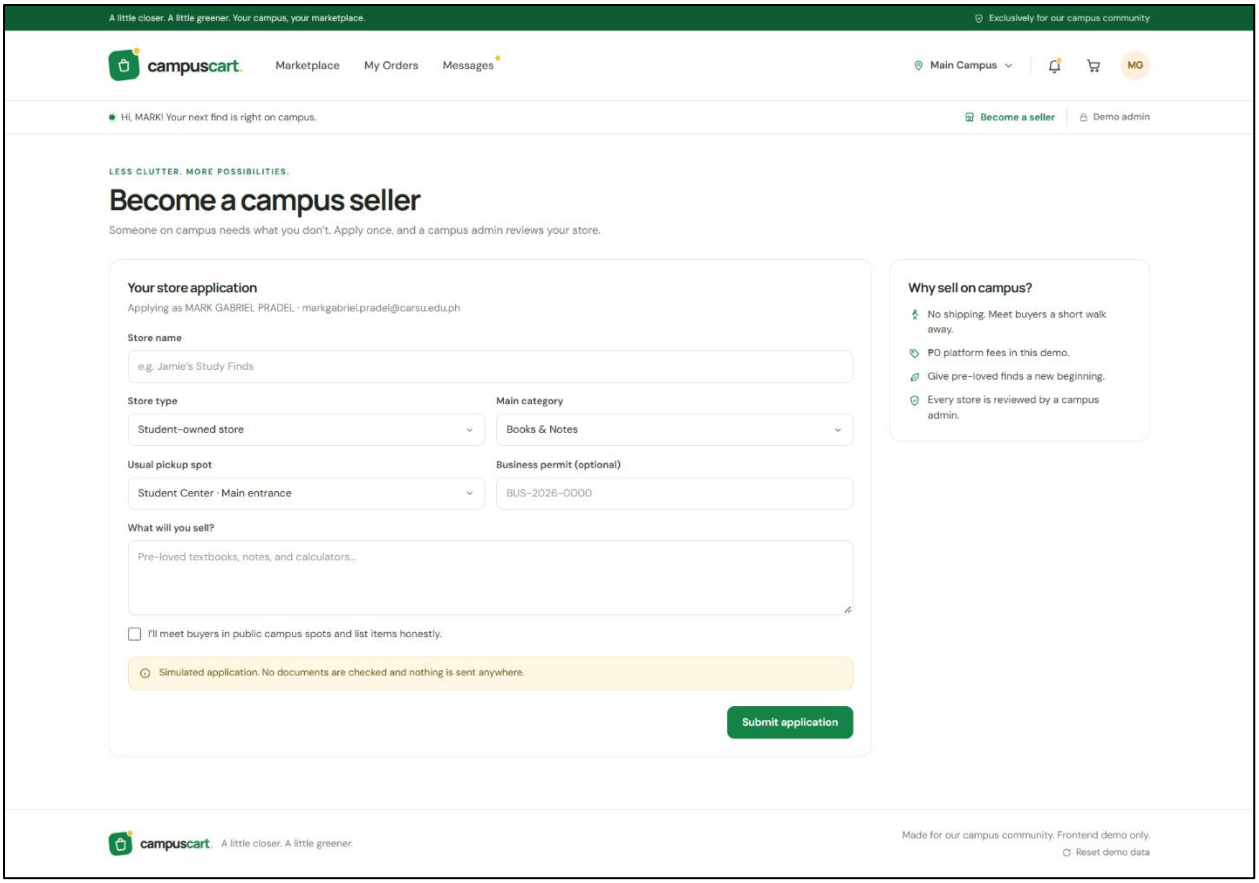


Figure 11. *Seller application form*

Seller dashboard. After approval, a Shopping / Selling switch appears below the header. The seller workspace has a sidebar (Dashboard, My Store, Products, Orders, Messages, Payouts, Reviews, Settings). The dashboard shows total products, pending orders, completed orders and store rating, plus recent orders and a "Running a little low" list of items with two or fewer in stock.

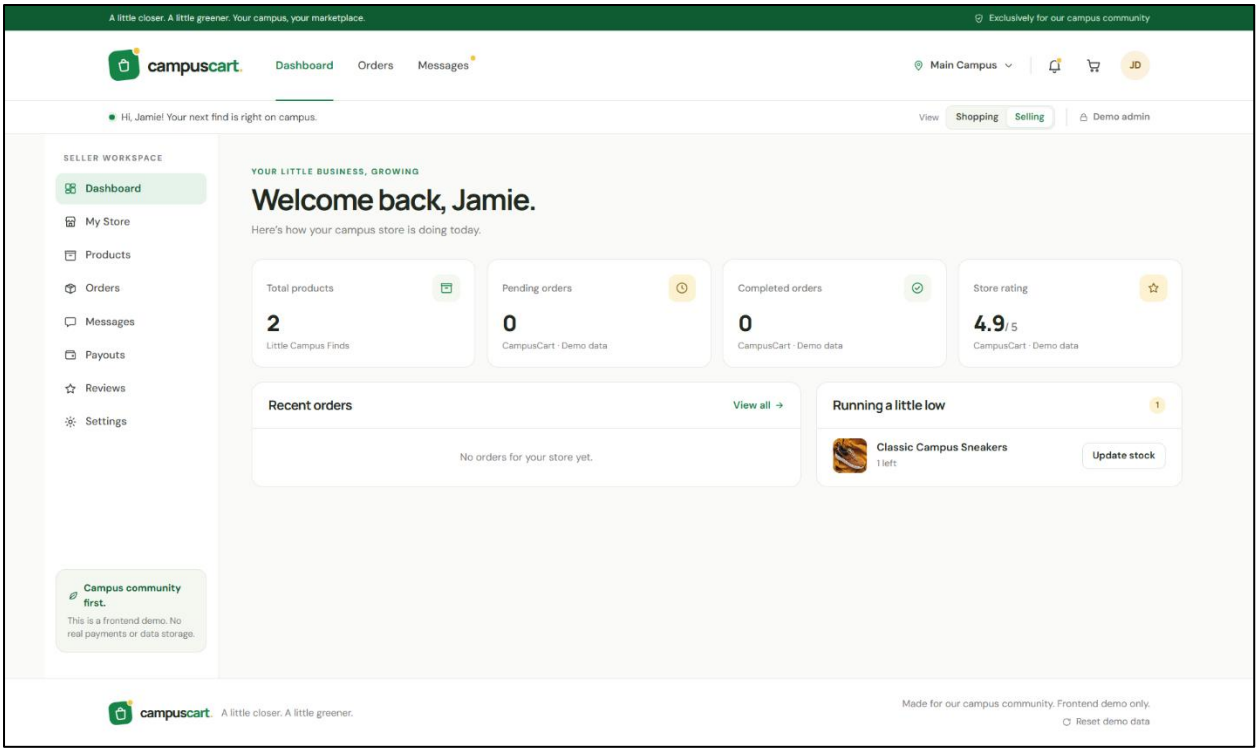


Figure 12. *Seller dashboard*

Product management and inventory. The Products page lists the seller's listings with price, stock and status (Published or Inactive). Edit opens the product dialog, and Deactivate sets stock to 0 while Publish sets it back to 1. Add product opens the same dialog for a new listing; selected photos are not uploaded, and a sample image is used.

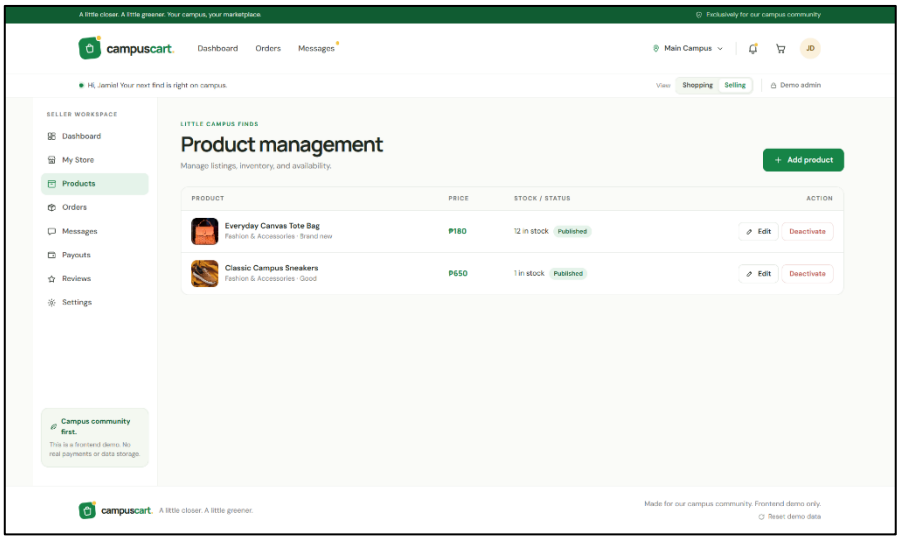


Figure 13. *Seller product management and add/edit product interface*

Store orders and settings. The seller's Orders page shows orders that contain the store's items, with Confirm order and Mark ready for pickup actions. My Store edits the store name, store type, campus and accepted payment methods. Payouts and Reviews display mock figures and sample reviews only.

D. Administrator Dashboard

The administrator uses a separate dashboard inside the same frontend application. It is opened through the **Demo admin** entry, not through public sign-in, and a bar under the header reads "Demo admin view · actions are simulated."

Dashboard overview and statistics. The Marketplace overview shows six statistic cards: total users, approved vendors, pending applications, active products, open reports and total orders. Total users and approved vendors add fixed mock numbers (2,478 and 46) to the demo counts. A "Needs your attention" panel links to pending applications and open reports.

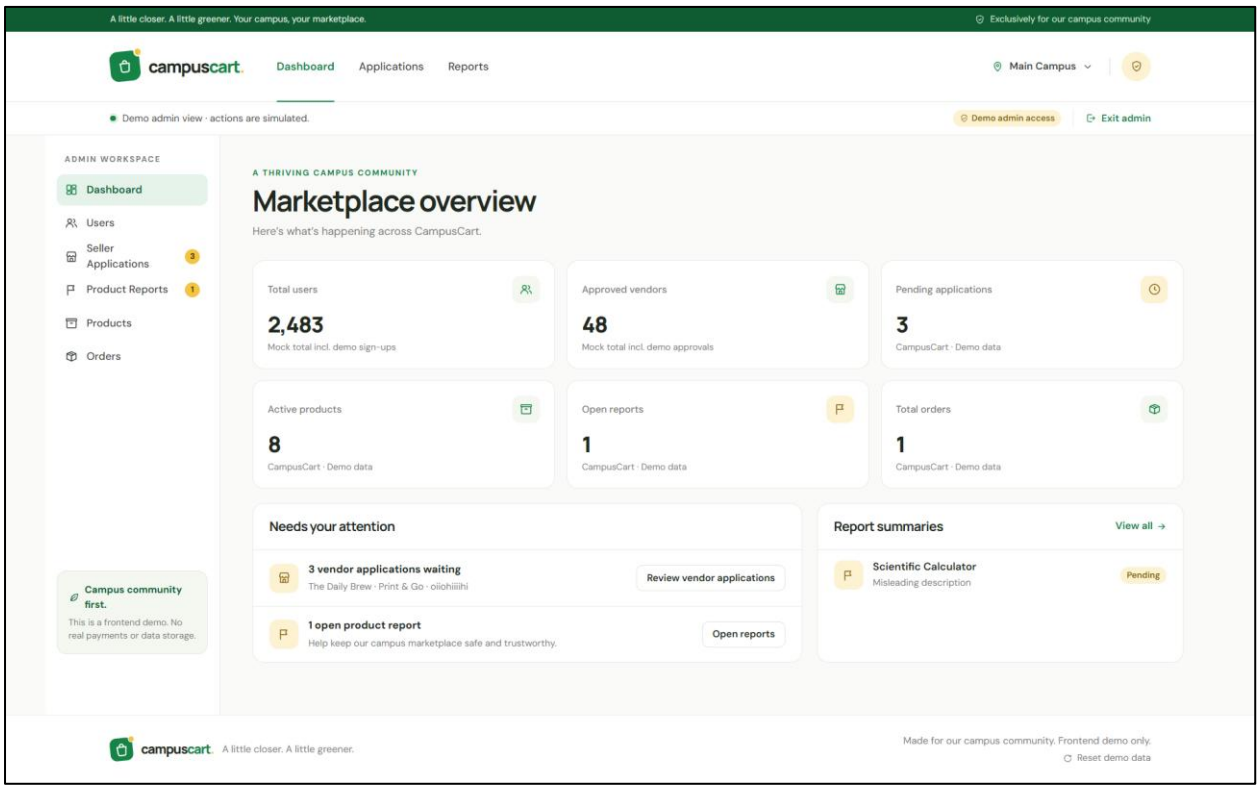


Figure 14. *Prototype administrator dashboard*

Seller application review. Seller Applications lists each application with store name, type, applicant, permit, category, pickup location and status. Pending applications have Approve vendor and Reject application buttons, each confirmed in a dialog.

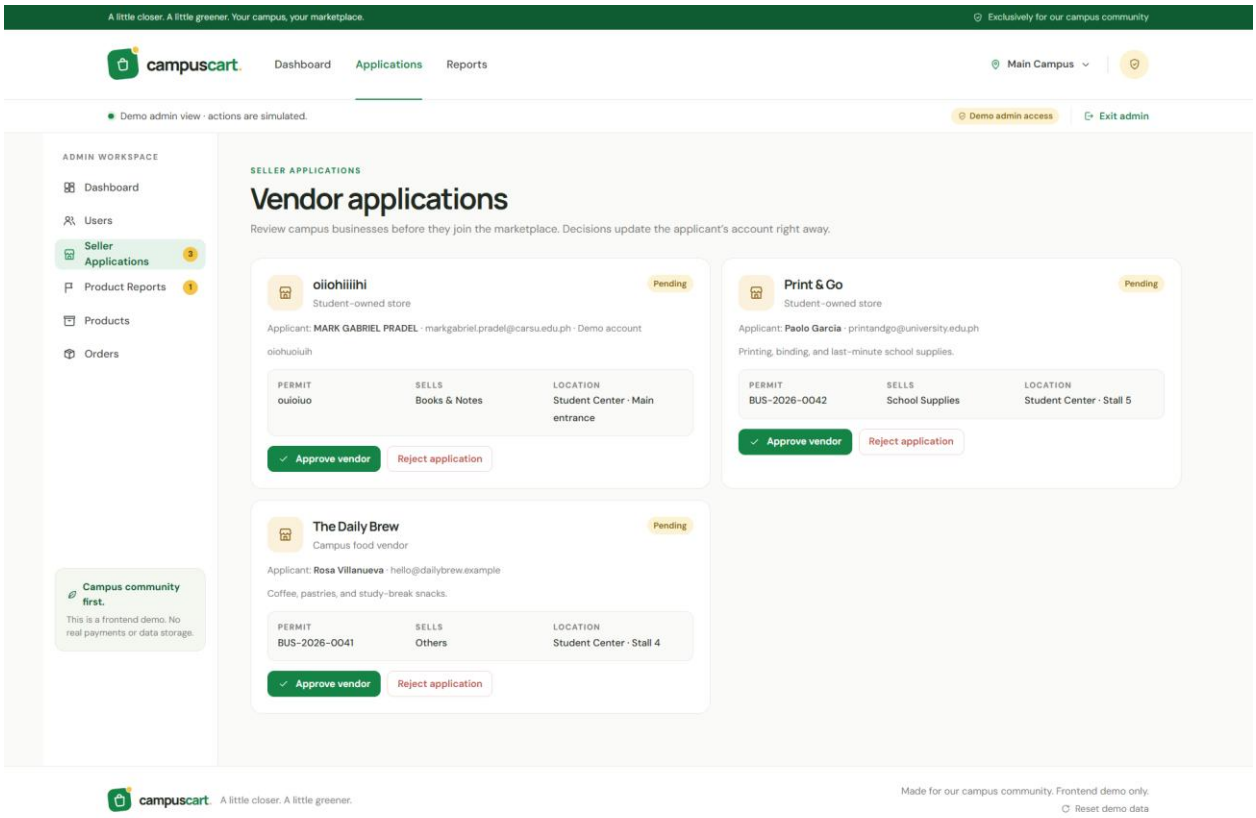


Figure 15. *Seller application review with simulated approval*

User management. Users lists every demo account with student number, department, campus, role (Buyer, Buyer · applied or Seller) and a Verified label, with search by name or email. This view is read-only; "View details" shows a short summary, and actions such as suspending accounts are not implemented.

Product reports, products and orders. Product Reports lets the administrator mark a report as under review or resolved. Products shows all listings in the same table used by sellers, with Edit and Deactivate or Publish. Orders shows every order in read-only mode.

Campus site or location management is not implemented. The header's campus picker shows only Main Campus ("Other campuses are coming soon in this demo"), and pickup locations are a fixed list in the sample data. This feature is listed in the plan for the Final.

E. Public Marketplace, Seller Dashboard and Admin Dashboard Compared

Table 11. *Distinction between the three interfaces*

	Public marketplace	Seller dashboard	Admin dashboard
Who uses it	Guests and buyers	Buyers with an approved seller application	The demo administrator

	Public marketplace	Seller dashboard	Admin dashboard
How it is opened	Default view when the app opens	Selling side of the Shopping / Selling switch	Demo admin entry with prefilled credentials
Layout	Full-width pages with header navigation	Workspace with a seller sidebar	Workspace with an admin sidebar
Main functions	Browse, search, product details, cart, checkout, orders, profile	Store statistics, products and stock, store orders, store settings, payouts, reviews	Statistics, seller applications, product reports, users, all products, all orders
Can shop?	Yes (after sign-in)	Yes, by switching back to Shopping	No; the admin must exit to browse as a buyer

F. Feature Status Summary

Table 12. *Feature and screen status*

Feature / Screen	Status	Notes
Marketplace home, search, categories, filters, sort	Done	Uses 8 sample products
Product details	Done	Description and comment are shared placeholder text
Sign in, create account, sign out	Done (simulated)	Browser-only accounts; no verification
Cart and checkout	Done (simulated)	No payment processing; orders split per store
Order history and order status flow	Done (simulated)	Cancel, confirm pickup, review
Profile and settings	Done	Saved in the browser
Messages and notifications	In progress	One sample conversation; messages are not delivered
Seller application and approval	Done (simulated)	Approved through the demo admin
Seller dashboard, products, inventory, store orders, store settings	Done	Photo upload not stored
Payouts and seller reviews	In progress	Mock figures only
Admin dashboard, applications, reports, products, orders	Done (simulated)	Some totals are mock numbers
Admin user management	In progress	View and search only
Campus site or location management	Not started	Only Main Campus is shown
URL routing (Vue Router)	Not started	State-driven page switching used instead
Pinia store	Not started	Composable store used instead

Feature / Screen	Status	Notes
Mock API / JSON Server integration	Not started	Local sample data and localStorage used instead
Automated tests	Not started	Type check and build only

- **Challenges encountered**
  - **Rebuilding the Figma Make design in Vue.** The prototype could not be exported directly into Vue components, so each screen was rebuilt by hand and compared against the design.
  - **Simulating roles without a backend.** One store holds the session and applies access rules for guests, buyers, approved sellers and the administrator. The administrator has a separate entry so it cannot be reached through sign-up.
  - **Not losing a guest's action at sign-in.** The store remembers the action a guest attempted (a "pending intent") and completes it once after sign-in, then forgets it if the dialog is closed.
  - **Keeping demo data reliable across refreshes.** Saved data is validated field by field on load, and a reset button restores the sample data.
  - **Keeping cart quantities realistic.** Quantities are capped at the seller's stock, and orders with items from several stores are split into one order per store.
- **Team contributions**

Table 13. *Team contributions*

Member	Contributions
Pradel	Designed the UI prototype in Figma Make; set up the Vue 3 + TypeScript + Vite project; built the central store (useMarketplace.ts) and the browser save and validation logic (persistence.ts); set up Vue Router with page URLs and access guards; built the public marketplace, product details, cart and checkout screens; built the simulated sign-in, registration and demo admin pages
Garcia	Built the seller application, the seller dashboard, product management, store orders, payouts and reviews; built the admin dashboard, seller application review, users and product reports screens; handled responsive layouts and accessibility; wrote the project documentation

V. Remaining Work and Plan for Final

Table 14. *Remaining work for the Final*

Task	Responsible member	Target date
Add Vue Router with page URLs and navigation guards for buyer, seller and admin pages	Pradel	November 6, 2026
Move the central store to Pinia	Pradel	November 13, 2026
Set up JSON Server (db.json) and read products, orders and applications through HTTP requests	Pradel	November 27, 2026
Integrate with backend endpoints for authentication and data, if provided by backend developers	Garcia	November 13, 2026
Implement campus site and pickup location management in the admin dashboard	Garcia	November 20, 2026
Add admin user actions (for example, verify or suspend an account)	Garcia	November 20, 2026
Product photo upload and preview; product-specific descriptions	Garcia	November 27, 2026
Messages per conversation with each seller	Pradel	December 4, 2026
Unit and UI tests with Vitest and Vue Test Utils	Pradel	December 4, 2026
Accessibility and cross-browser checks (Lighthouse or axe); darken the muted text color	Garcia	December 11, 2026
Publish the GitHub repository and deploy online (Netlify or Vercel)	Pradel	December 11, 2026

Appendix

- **Links**

Table 15. *Project links*

Item	Link or location
GitHub repository	https://github.com/markpradel8756/campuscart.git
Design file	https://www.figma.com/make/Kjc0EIqjNk1XUBBDiQ2qYP/CampusCart-Prototype?t=FA4w5I5plkRI21JH-1

- **Running the prototype locally**

  Requires Node.js 18 or newer. In the project folder, run `npm install` and then `npm run dev`, and open http://localhost:5173 in a browser. `npm run build` type-checks and builds the project, and `npm run preview` serves the built version.

- **Demo accounts**

Table 16. *Demo accounts for the prototype*

Who	How to get in
Guest	Open the app. Browsing, search, filters and product details need no account.
Buyer and approved seller	`jamie.delacruz@university.edu.ph` / `campus123` (the sign-in dialog has a "Use it" shortcut)
Other buyers	`miguel.santos@university.edu.ph` or `andrea.reyes@university.edu.ph` / `campus123`
New buyer	Create an account with any school or personal email; new accounts are always buyers
Demo administrator	Select Demo admin under the header; `admin@campusmarket.demo` / `admin-demo` are prefilled